

Ngôn ngữ PHP

làm việc với các biến siêu toàn cục

NGUYỄN THỊ THÙY LIÊN

KHOA CNTT-ĐH PHENIKAA

LIEN.NGUYENTHITHUY@PHENIKAA-UNI.EDU.VN

Các biến siêu toàn cục trong PHP

- `$GLOBALS`
- `$_SERVER`
- `$_REQUEST`
- `$_POST`
- `$_GET`
- `$_FILES`
- `$_ENV`
- `$_COOKIE`
- `$_SESSION`

PHP \$GLOBALS

- Sử dụng để truy cập các biến toàn cục từ bất cứ vị trí nào
- Chứa các phần tử gồm: \$_GET, \$_POST, \$_COOKIE, \$_FILES, và các biến toàn cục người dùng khai báo.

```
<?php
    $a = 5; $b = 6;
    function tinhtong() {
        $GLOBALS['c'] = $GLOBALS['a'] + $GLOBALS['b'];
    }
    tinhtong();
    echo $c;
?>
```

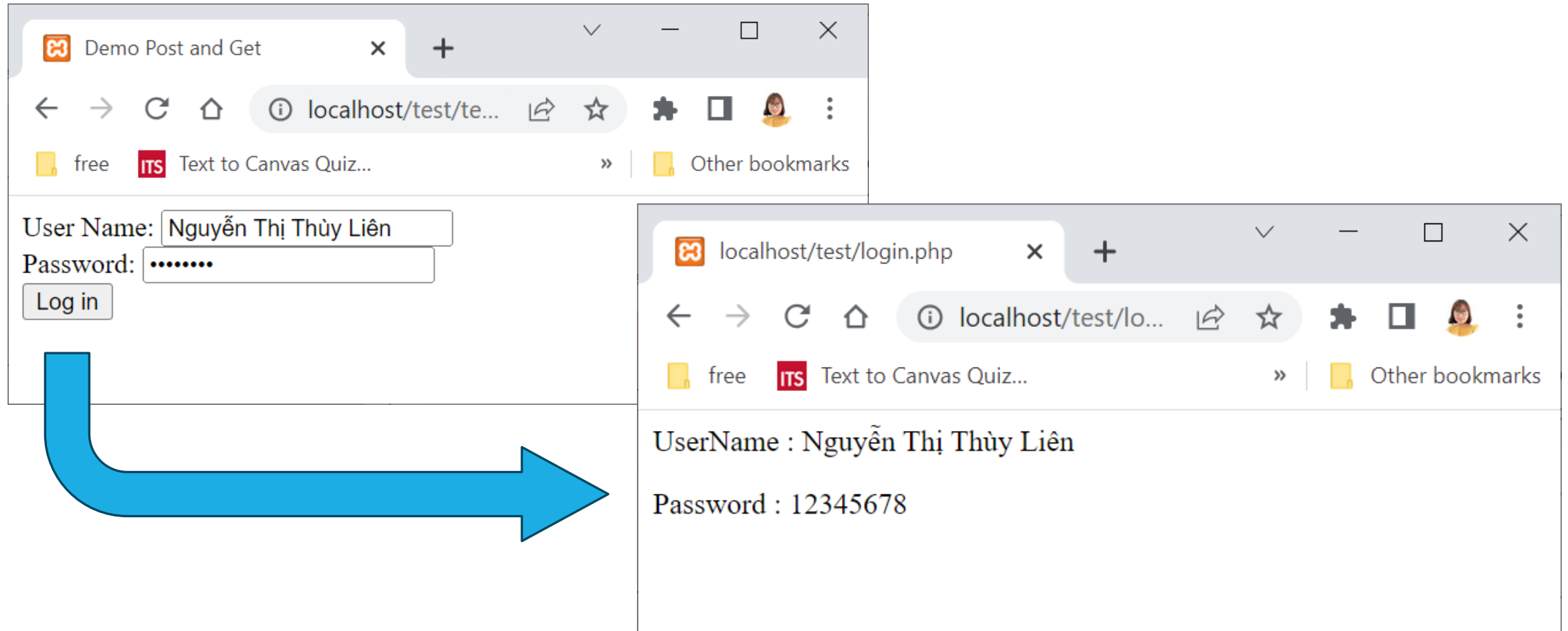
PHP \$_SERVER

- Sử dụng để trả về thông tin tiêu đề, vị trí đường dẫn và vị trí tệp tin.
 - \$_SERVER ['PHP_SELF']: in hoặc trả về tên tệp hiện tại
 - \$_SERVER ['SERVER_NAME']: tên máy chủ hiện tại
 - \$_SERVER ['SERVER_ADDR']: trả về địa chỉ Giao thức Internet (IP) của máy chủ lưu trữ.
 - \$_SERVER ['REQUEST_METHOD']: để trả về phương thức đã được sử dụng trong chương trình để gửi tức là GET hoặc POST.
 - \$_SERVER ['HTTP_USER_AGENT']: thông tin trình duyệt web
 - \$_SERVER ['SCRIPT_NAME']: trả về đường dẫn của tập lệnh hiện tại.

Truyền nhận dữ liệu từ HTTP

`$_REQUEST, $_POST, $_GET`

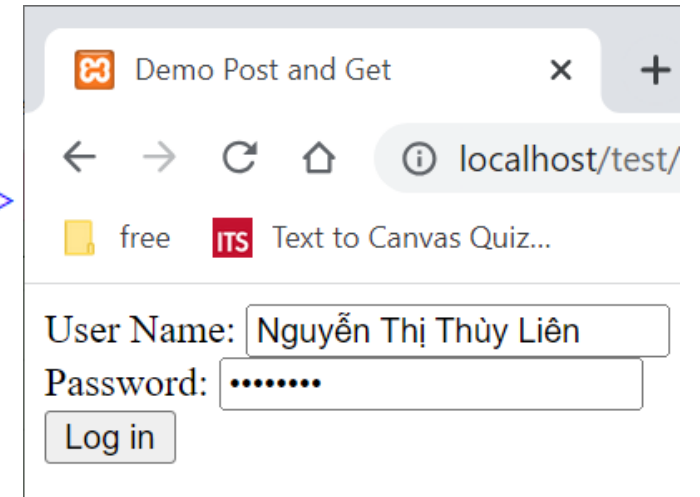
Cơ chế truyền nhận dữ liệu



Truyền dữ liệu

- Sử dụng đối tượng <form>
- Nhập liệu thông qua các đối tượng điều khiển trên form
- Thực hiện truyền dữ liệu thông qua submit

```
1 <html>
2   <head></head>
3   <title>Demo Post and Get</title>
4   <body>
5     <form name="frmLogin" action="login.php" method="post">
6       <label>User Name: </label>
7       <input type="text" name="txtUserName"/><br/>
8       <label>Password: </label>
9       <input type="password" name="txtPassword" /><br/>
10      <input type="submit" value="Log in"/><br/>
11    </form>
12  </body>
13 </html>
```



Demo Post and Get

localhost/test/

free ITS Text to Canvas Quiz...

User Name: Nguyễn Thị Thùy Liên

Password:

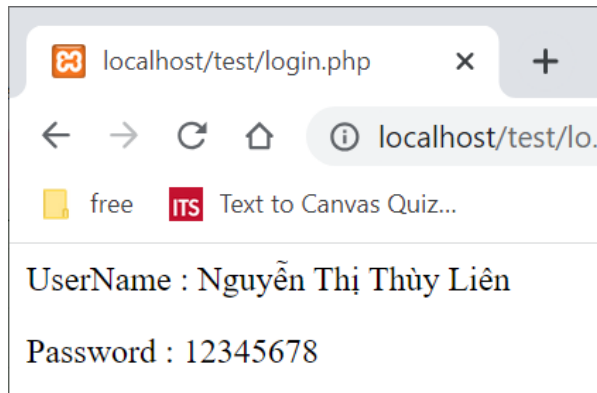
Log in

Truyền dữ liệu

- **<form>** trong HTML có 2 thuộc tính:
 - **action="chuỗi"**: Địa chỉ trang web nhận dữ liệu (địa chỉ file PHP xử lý dữ liệu)
 - **method="chuỗi"**: Phương thức gửi dữ liệu
 - GET (mặc định)
 - POST

Nhận dữ liệu

- Nhận dữ liệu từ các yêu cầu HTTP thông qua các biến mảng:
 - \$_POST
 - \$_GET
 - \$_REQUEST



```
1 <html>
2   <head>
3   </head>
4   <body>
5       <?php
6           $username = $_POST['txtUserName'];
7           $password = $_POST['txtPassword'];
8           echo "<p>UserName : ".$username."</p>";
9           echo "<p>Password : ".$password."</p>";
10      ?>
11   </body>
12 </html>
```

Truyền nhận với phương thức GET (URL)

- Truyền dữ liệu (biến) qua URL:
 - Các biến được truyền thành từng cặp **biến=giá_trị** phân cách bởi dấu **&**
 - Phân cách với địa chỉ trang ban đầu bởi dấu hỏi chấm (?)
- Ví dụ: Truyền 3 biến a, b, c có giá trị lần lượt là 1, 2, -3 vào trang <http://localhost/ptb2.php> qua URL:
- <http://localhost/ptb2.php?a=1&b=2&c=-3>
- Tối đa 2000 ký tự

Truyền nhận với phương thức GET (URL)

- Để đọc giá trị các biến trong PHP: Sử dụng mảng **\$_GET**, **\$_REQUEST**, các chỉ số là tên biến.
- Ví dụ: Trong trang ptb2.php ở trên đọc các biến a, b, c:

```
$a = $_GET["a"];
```

```
$b = $_GET["b"];
```

```
$c = $_GET["c"];
```

```
$a = $_REQUEST["a"];
```

```
$b = $_REQUEST["b"];
```

```
$c = $_REQUEST["c"];
```

Truyền nhận với phương thức GET (FORM)

- Khi Submit 1 form sử dụng phương thức GET, dữ liệu được truyền qua URL:
 - Tên các biến là tên đối tượng trên form
 - Giá trị biến là giá trị NSD nhập vào đối tượng

```
5  
6  
7  
8  
9  
10  
11
```

```
<form name="frmLogin" action="login.php" method="GET">  
  <label>User Name: </label>  
  <input type="text" name="txtUserName"/><br/>  
  <label>Password: </label>  
  <input type="password" name="txtPassword" /><br/>  
  <input type="submit" value="Log in"/><br/>  
</form>
```

① localhost/test/login.php?txtUserName=Nguyễn+Thị+Thùy+Liên&txtPassword=12345678

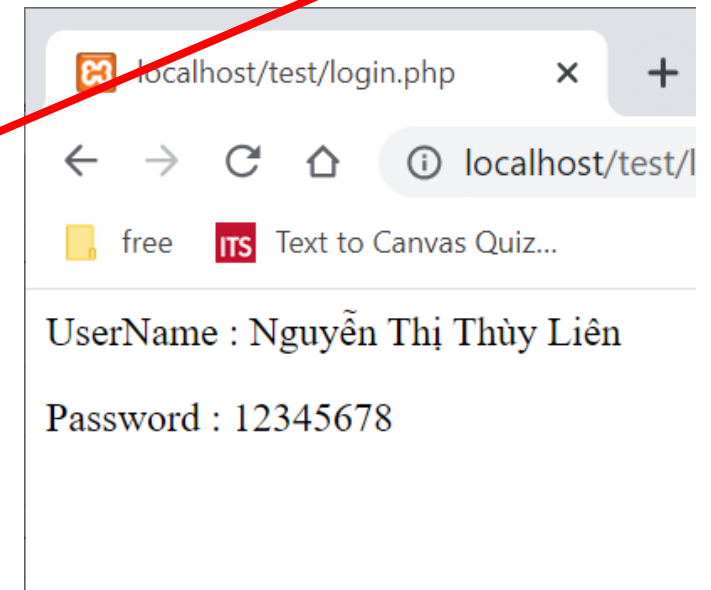
Truyền nhận với phương thức GET (URL)

- Nhận dữ liệu ở file php xử lý dữ liệu (login.php)

① localhost/test/login.php?txtUserName=Nguyễn+Thị+Thùy+Liên&txtPassword=12345678

Login.php

```
<?php
$username = $_GET['txtUserName'];
$password = $_GET['txtPassword'];
echo "<p>UserName : ".$username."</p>";
echo "<p>Password : ".$password."</p>";
?>
```



Truyền dữ liệu theo phương thức POST (FORM)

- Khi Submit 1 form sử dụng phương thức POST
- Dữ liệu của Form post được truyền trong thân của yêu cầu HTTP
- Không hạn chế kích thước dữ liệu truyền

```
<form name="frmLogin" action="login.php" method="POST">  
  <label>User Name: </label>  
  <input type="text" name="txtUserName"/><br/>  
  <label>Password: </label>  
  <input type="password" name="txtPassword" /><br/>  
  <input type="submit" value="Log in"/><br/>  
</form>
```

Truyền dữ liệu theo phương thức POST (FORM)

The screenshot illustrates a web browser at the URL `localhost/test/login.php`. The browser's address bar and the form's `action` attribute are circled in red, with a red arrow pointing from the address bar to the `login.php` part of the `action` attribute. The form contains two input fields: "UserName : Nguyễn Thị Thùy Liên" and "Password : 12345678".

The HTML code for the form is shown in a box:

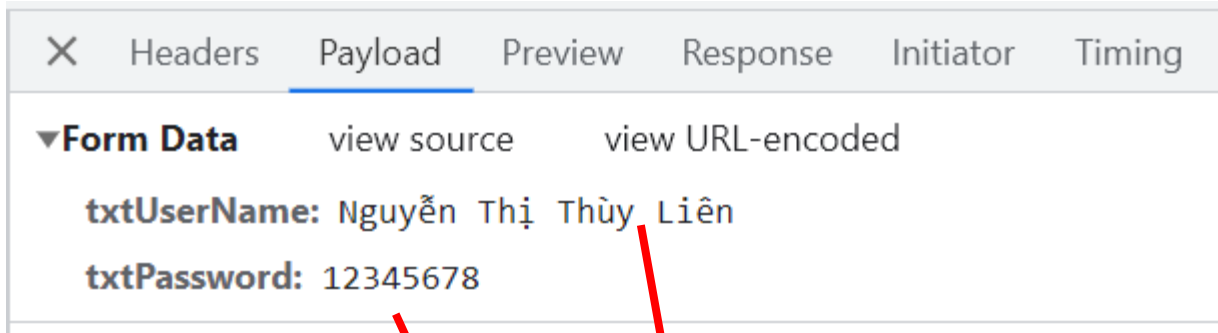
```
<form name="frmLogin" action="login.php" method="POST">
  <label>User Name: </label>
  <input type="text" name="txtUserName"/><br/>
  <label>Password: </label>
  <input type="password" name="txtPassword" /><br/>
  <input type="submit" value="Log in"/><br/>
</form>
```

The browser's developer tools are open to the Network tab. The list of requests shows `login.php` and `favicon.ico`. The `login.php` request is selected, and the "Form Data" tab is active, showing the submitted data:

Name	Value
txtUserName	Nguyễn Thị Thùy Liên
txtPassword	12345678

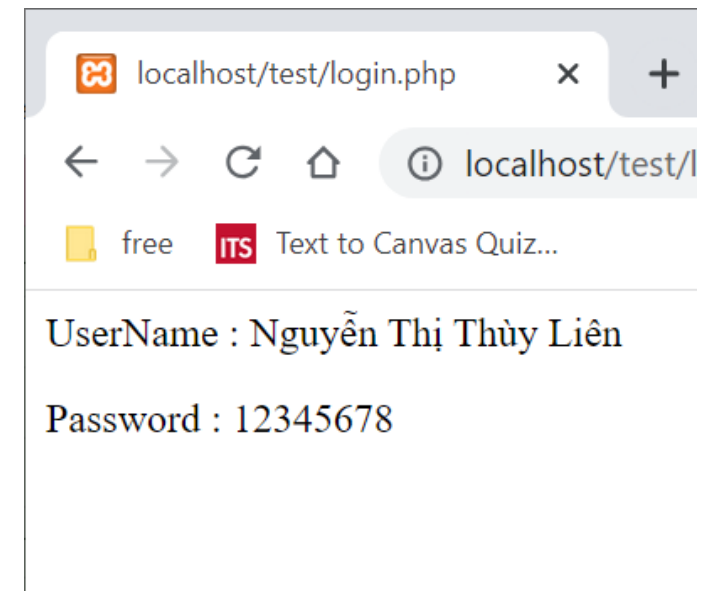
Truyền dữ liệu theo phương thức POST

- Nhận dữ liệu qua mảng **`$_POST`**, **`$_REQUEST`** với các chỉ số là tên của đối tượng trên form gửi đến.



Login.php

```
<?php
$username = $_POST['txtUserName'];
$password = $_POST['txtPassword'];
echo "<p>UserName : ".$username."</p>";
echo "<p>Password : ".$password."</p>";
?>
```



Trường hợp đặc biệt

- Nhận dữ liệu gửi từ các **checkbox** cùng tên:

- **Gui.html**

```
<form action=Nhan.php method=post>
    <input type=checkbox name=box[] value=1>Một
    <input type=checkbox name=box[] value=2>Hai
    <input type=checkbox name=box[] value=3>Ba
    <input type=submit name="Sumit">
</form>
```

- **Nhan.php**

```
<?php
    $val = $_POST['box']; // $val là mảng các phần tử
    foreach($val as $k=>$v) echo $v."<br>";
?>
```

So sánh GET vs POST

GET	POST
• Dữ liệu hiển thị toàn bộ trên URL • Có yêu cầu nhận dữ liệu từ máy chủ CSDL • Muốn thực thi yêu cầu nhiều lần mà không gây lỗi • Bookmark (đánh dấu) trang kết quả trả về	• Bảo mật hơn • Có yêu cầu ghi dữ liệu lên máy chủ CSDL • Việc thực hiện nhiều gây ảnh hưởng tới trang • Không muốn truyền tham số • Cần truyền nhiều hơn 4KB dữ liệu

Cookie & Session

`$_COOKIE, $_SESSION`

Cookie

- Khái niệm
- Khai báo cookie
- Sử dụng cookie
- Xóa cookie

Cookie – Khái niệm

- Cookie là một tập tin nhỏ được trang web gửi đến máy người dùng và được lưu lại thông qua trình duyệt khi người dùng truy cập trang web đó
- Website sử dụng cookie để lưu trữ các thông tin người dùng
- Mỗi khi gửi request lên server, client đồng thời gửi cookie đã lưu lần trước lên server

Ứng dụng cookie

- Đếm số lần người dùng truy cập website
- Số người truy cập mới
- Số người truy cập thông thường
- Tần số truy cập website
- Lưu trữ thời gian mà người dùng truy cập website lần cuối
- Lưu trữ các thông tin cá nhân cho việc thiết lập trang web của người dùng: ghi nhớ mật khẩu, ...

Thiết lập cookie

- `setcookie(name, value, expire, path, domain, secure, httponly);`
 - name: Tên cookie được tạo ra
 - value: giá trị được đặt cho cookie
 - expire: Số: thời gian hết hạn của cookie
 - Path: đường dẫn

```
1 <?php
2     $cookie_name = "user";
3     $cookie_value = "LienNTT";
4     setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/");
5 ?>
```

Thiết lập cookie

- Chú ý:
 - Lệnh **setcookie** phải được gọi trước khi gửi bất cứ nội dung gì về client (Trước các thẻ HTML, trước echo, print)
 - Để thiết lập thời gian hết hạn của cookie thường sử dụng hàm `time()+khoảng thời gian (tính bằng giây)`
 - giá trị của biến cookie sẽ tự động được URL mã hóa khi gửi cookie đi, và tự động giải mã khi nhận cookie về. (Nếu không muốn URL mã hóa thì dùng hàm `setrawcookie()`)

Thiết lập cookie

- Web server gửi cookie trong một trường Set-Cookie header

▼ Response Headers	☐ Raw
Connection:	Keep-Alive
Content-Length:	328
Content-Type:	text/html; charset=UTF-8
Date:	Tue, 27 Jun 2023 08:22:28 GMT
Keep-Alive:	timeout=5, max=100
Server:	Apache/2.4.46 (Win64) OpenSSL/1.1.1g PHP/7.2.34
Set-Cookie:	user=LienNTT; expires=Thu, 27-Jul-2023 08:22:28 GMT; Max-Age=2592000; path=/
X-Powered-By:	PHP/7.2.34

- Set-Cookie là một phần của gói thông tin HTTP response

Lấy giá trị cookie

- Mỗi khi gửi request lên server, client đồng thời gửi cookie đã lưu lần trước lên server trong trường cookie của HTTP

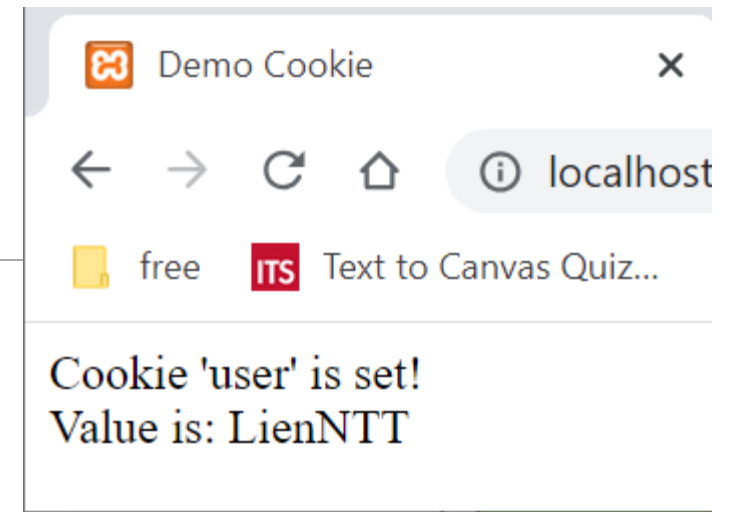
request:

▼ Request Headers	☐ Raw
Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding:	gzip, deflate, br
Accept-Language:	vi-VN,vi;q=0.9,fr-FR;q=0.8,fr;q=0.7,en-US;q=0.6,en;q=0.5
Cache-Control:	no-cache
Connection:	keep-alive
Cookie:	user=LienNTT
Host:	localhost
Pragma:	no-cache

Lấy giá trị cookie

- Sử dụng **\$_COOKIE** :
- Chú ý:
 - Ta không thể đọc cookie vừa được thiết lập ngay trong cùng 1 trang vừa thiết lập gọi `setcookie`.

```
if(!isset($_COOKIE[$cookie_name])) {  
    echo "Cookie named '" . $cookie_name . "' is not set!";  
} else {  
    echo "Cookie '" . $cookie_name . "' is set!<br>";  
    echo "Value is: " . $_COOKIE[$cookie_name];  
}
```



Xóa cookie

- Tương tự như việc thiết lập cookie
- Có hai cách để xóa cookie:
 - Thiết lập lại thời gian hết hạn của cookie thành 1 thời điểm trong quá khứ
setcookie ("{\$cookie_name}", "", *time()-8000*);
 - Thiết lập lại giá trị của cookie thông qua tên của cookie
setcookie(\$cookie_name);

Nhược điểm

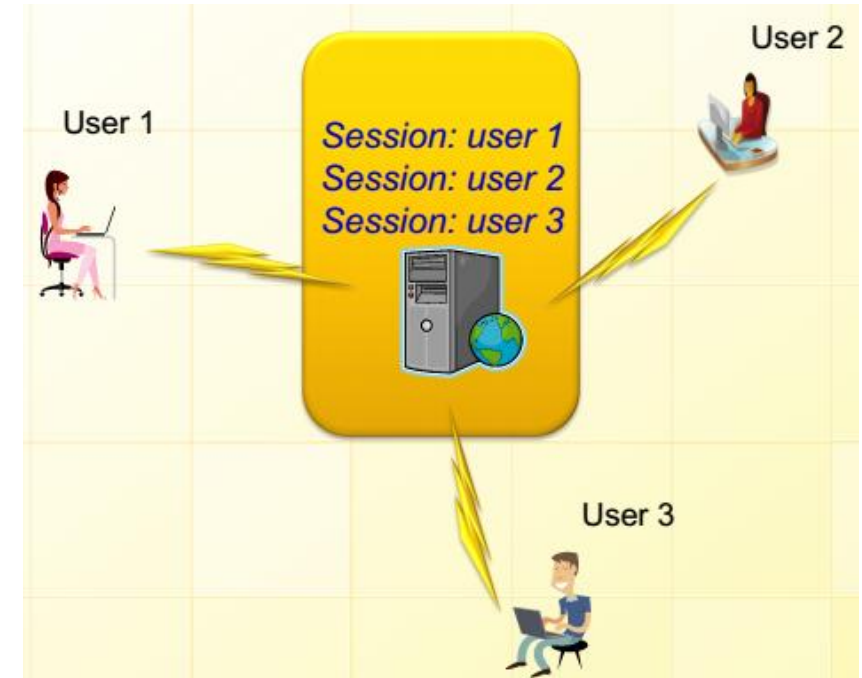
- Cookie không bảo mật và độ tin cậy không cao
- Độ dài thông tin lưu trữ trong cookie là hữu hạn
- Càng nhiều cookie được lưu trữ sẽ càng làm máy tính chậm hơn.
- Người dùng có thể khóa việc lưu cookie vào ổ cứng
- Dễ xảy ra nhầm lẫn và ghi đè khi có nhiều người dùng sử dụng cùng nhau

Session

- Khái niệm
- Cách thức hoạt động
- Khởi động session
- Đăng ký session
- Sử dụng session
- Hủy biến session

Session

- Thông tin người dùng lưu trữ cho một website cụ thể
- Thông tin được lưu trữ trong suốt khoảng thời gian sử dụng website
- Cho phép phân biệt người dùng khác nhau truy cập website



Ứng dụng Session

- Tạo ứng dụng giỏ hàng, đăng nhập...
- Đếm số người dùng đang online...
- Truyền giá trị từ trang này sang trang khác
- Thay thế cho cookie khi trình duyệt không hỗ trợ cookie

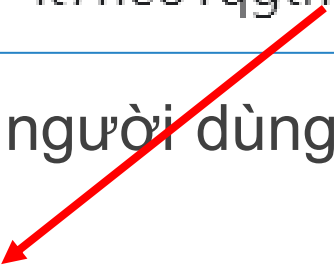
Cách thức hoạt động

- Session xuất hiện khi một người dùng truy cập một website, PHP tạo ra một định danh duy nhất cho session đó, chuỗi kí tự ngẫu nhiên 32 số hexa.
- Một Cookie được gọi là PHPSESSID được gửi tự động từ server đến máy tính người dùng để lưu trữ chuỗi định danh session.

Cookie: user=LienNTT; PHPSESSID=it7he81qgtm8kkn3ie7misa992

- Một file session tương ứng lưu trữ thông tin người dùng sẽ được tạo ra trên server

 sess_it7he81qgtm8kkn3ie7misa992



Cách thức hoạt động

- Khi PHP script muốn lấy giá trị từ một biến session, PHP sẽ lấy chuỗi định danh PHPSESSID từ cookie và tìm file session trên server.
- Một session sẽ kết thúc khi người dùng tắt trình duyệt hoặc sau khi rời khỏi site.
- Server sẽ chấm dứt session sau một thời gian định trước, thường là 30 phút.

Khởi tạo session

- Hàm **session_start()** cho phép khởi tạo một session.
 - Phải được đặt trên đầu của trang web hoặc trước tất cả các mã PHP
 - Hàm luôn trả về true

```
<?php
    // bắt đầu session
    session_start();
?>
```

Sử dụng session

- Dùng biến **\$_SESSION**

```
1  <?php
2      session_start();
3      $_SESSION["user_id"] = "LienNTT";
4
5
6      if(isset($_SESSION["user_id"])) {
7          echo "Xin chào: " . $_SESSION["user_id"];
8      }else{
9          echo "Chưa khởi tạo session";
10     }
11
12  ?>
```

Kết thúc session

- Để kết thúc session:
 - session_unset()***: Xóa tất cả các biến session đã tạo
 - session_destroy()***: Xóa toàn bộ session
 - unset()***: Xóa một biến
- Hàm sẽ xóa tất cả file session tương ứng trên hệ thống
- \$PHPSESSID cookie sẽ không bị xóa từ Web browser

```
<?php
    session_unset();
    session_destroy();
?>
```

COOKIE & SESSION

Cookies	Sessions
Lưu trữ thông tin phía client	Lưu trữ thông tin phía server
Thông tin có thể vẫn tồn tại khi kết thúc web browser	Thông tin sẽ bị xóa sau khi kết thúc web browser
Người dùng có thể khóa cookie	Người dùng không thể khóa session
Có kích thước giới hạn(4KB)	Kích thước không giới hạn

Upload file lên Server

`$_FILES`

Upload File lên server

- Kiểm tra cấu hình **file_uploads = On** trên file **php.ini**
- Form upload:
 - Luôn sử dụng phương thức POST
 - Luôn sử dụng thuộc tính **enctype = "multipart/form-data"**

```
<form action="upload.php" method="post" enctype="multipart/form-data">  
  File upload:  
  <input type="file" name="newfile" />  
  <input type="submit" value="Upload" />  
</form>
```

File upload: No file chosen

Upload file lên Server

Biến \$_FILES	
<code>\$_FILE['newfile']['name']</code>	Tên file được upload
<code>\$_FILE['newfile']['type']</code>	Loại file được upload (imag/gif, image/jpeg,...)
<code>\$_FILE['newfile']['size']</code>	Kích thước file được upload (byte)
<code>\$_FILE['newfile']['tmp_name']</code>	Vị trí file được lưu tạm trên server
<code>\$_FILE['newfile']['error']</code>	Mã lỗi của việc upload (0: upload thành công)

Upload file lên Server

```
<?php
    $dir = "./"; //Upload vào thư mục hiện tại

    if($_FILES['newfile']['name'] != "")
    {
        $fileupload = $dir . $_FILES['newfile']['name'];
        if(move_uploaded_file($_FILES['newfile']['tmp_name'], $fileupload))
        {
            echo "Upload file thành công!";
        }
        else{
            echo "Upload file không thành công!";
        }
    }
    else{
        echo "Vui lòng chọn file để upload!";
    }
?>
```

Upload.php:

Upload file lên Server

- Upload nhiều file

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  Files upload:<br />
  File 1: <input type="file" name="newfile[]" /><br />
  File 2: <input type="file" name="newfile[]" /><br />
  File 3: <input type="file" name="newfile[]" /><br />
  <input type="submit" value="Upload" />
</form>
```

Files upload:

File 1:	<input type="file"/>	No file chosen
File 2:	<input type="file"/>	No file chosen
File 3:	<input type="file"/>	No file chosen
<input type="submit" value="Upload"/>		

Upload file lên Server

- Xử lý upload nhiều file:

```
<?php
    foreach($_FILES['newfile']['error'] as $key => $error)
    {
        if($error == 0)
        {
            $tmp_name = $_FILES['newfile']['tmp_name'][$key];
            $name = $_FILES['newfile']['name'][$key];
            move_uploaded_file($tmp_name, ".$name");
        }
    }
?>
```

Làm việc với File

Làm việc với file

- Mở file:
 - Cú pháp: **fopen**("Đường dẫn", thuộc tính)
 - Trong đó: thuộc tính bao gồm các quyền hạn cho phép thao tác trên file đó
- Đóng file:
 - Cú pháp: **fclose**(\$file)
- Ví dụ:

```
$file = fopen("text.txt",r) or exit ("không tìm thấy")  
// xử lý file ở đây  
fclose($file);
```

Chế độ làm việc với file

Mode	Ý nghĩa
r	Mở file để đọc
w	Mở file để ghi, tạo mới file nếu chưa có, ghi đè nếu file đã tồn tại
a	Mở file để ghi thêm, tạo file mới để ghi nếu chưa tồn tại
x	Mở file để ghi, trả về false hoặc lỗi nếu file đã tồn tại
r+	Mở file để đọc ghi. Con trỏ file bắt đầu ở đầu file
w+	Mở file để đọc ghi. Tạo mới file nếu chưa có, Ghi đè nếu file đã tồn tại
a+	Mở file để đọc ghi. Ghi thêm vào cuối file, tạo mới nếu file chưa tồn tại
x+	Mở file để đọc ghi. Trả về false hoặc lỗi nếu file đã tồn tại

Đọc file trong php

- Đọc file kích thước:
 - Cú pháp: **fread**(\$file,\$size)
 - \$file: tên file, \$size: kích thước tối đa để đọc

```
<?php
```

```
    $fp=fopen("test.txt",r) or exit("khong tim thay file ");
```

```
    echo fread($fp,filesize("test.txt")) ;
```

```
    fclose($fp) ;
```

```
?>
```


Đọc file trong php

- Đọc file theo từng dòng:

- Cú pháp: **fgets**(\$file)

```
<?php
```

```
    $fp=fopen("test.txt",r) or exit("khong tim thay file ");
```

```
    echo fgets($fp) ;
```

```
    fclose($fp) ;
```

```
?>
```

- Đọc file theo từng ký tự:

- Cú pháp: **fgetc**(\$file)

```
<?php
```

```
    $fp=fopen("test.txt",r) or exit("khong tim thay file ");
```

```
    echo fgetc($fp) ;
```

```
    fclose($fp) ;
```

```
?>
```

Đọc file trong php

- Cú pháp kiểm tra kết thúc file:
 - **feof**(\$file)

- Ví dụ:

```
<?php
```

```
    $fp=fopen("test.txt",r) or exit("khong tim thay file ");
```

```
    while (!feof($fp)) {
```

```
        echo fgets($fp);
```

```
    }
```

```
    fclose($fp);
```

```
?>
```

Ghi file trong PHP

- Cú pháp: **fwrite**(\$file, \$content)

- Trong đó:

- \$file: file cần ghi
- \$content: nội dung dữ liệu cần ghi

- Ví dụ:

```
<?php
```

```
    $fp=fopen("test.txt",a) or exit("khong tim thay file ");
```

```
    $content="Noi dung can ghi\n";
```

```
    fwrite($fp,$content);
```

```
    fclose($fp);
```

```
?>
```

PHP & JSON

JSON

- JSON = JavaScript Object Notation
- Mẫu/cú pháp để lưu trữ dữ liệu giúp thuận tiện trao đổi dữ liệu
- Cú pháp:
{ “key” : “value” }

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

Mã hóa về dạng JSON

- Cú pháp: `json_encode($array)`

```
<?php
    $age = array("Peter"=>35, "Ben"=>37, "Joe"=>43);

    echo json_encode($age);
?>
```

String: {"Peter":35,"Ben":37,"Joe":43}

Giải mã chuỗi JSON

- Cú pháp: **json_decode(\$jsonObjectString)**
- Return: object hoặc associative array

```
<?php
    $jsonobj= ' {"Peter":35,"Ben":37,"Joe":43} ';

    $object = json_decode($jsonobj [,true]);
?>
```

PHP Object: {"Peter"=>35, "Ben"=>37, "Joe"=>43}
Array: ["Peter"=>35, "Ben"=>37, "Joe"=>43]

Lambda và Callback functions

Lambda – hàm ẩn danh

- Là các hàm ẩn danh, sử dụng một lần và có thể được định nghĩa bất cứ lúc nào
- Gắn với một biến hoặc gán vào một hàm như tham số
- Chỉ tồn tại trong phạm vi của biến mà nó được định nghĩa hoặc được gọi một lần trong hàm khi nó được khai báo như tham số.

```
<?php
    function () {
        return "Hello world";
    }
?>
```

Sử dụng hàm ẩn danh

- Gán vào biến và gọi hàm qua biến

```
<?php
    $hello = function () {
        return "Hello world";
    }

    echo $hello();
?>
```

Sử dụng hàm ẩn danh

- gán vào một hàm khác như tham số

```
<?php
    function display($mess){
        echo $mess;
    }
    display( function () {
        return "Hello world";
    });
?>
```

Callback function

- Là hàm được truyền vào như là một đối số (một chuỗi là tên hàm) khi gọi một hàm khác.

```
function sayHello($callback) {  
    echo "Xin chào!<br>";  
    $callback();  
}  
function sayGoodbye() {  
    echo "Tạm biệt!";  
}  
sayHello('sayGoodbye');
```